

Orale ing del software

Leonardo Ascenzi

January 20, 2026

Contents

1	Significato include/extend negli use case	4
1.1	Include:	4
1.2	Extend:	4
2	Requisiti Software	4
2.1	Requisiti utente:	4
2.2	Requisiti di sistema:	4
2.2.1	Funzionali:	4
2.2.2	Non funzionali:	5
2.2.3	Dominio:	5
2.3	Differenze:	5
3	Modello ciclo di vita	5
3.1	Principali modelli	5
3.1.1	Build & Fix	5
3.1.2	Waterfall	5
3.1.3	Spirale	5
4	Risk Analysis	6
4.1	Fasi principali:	6
5	Architettura softwares - Architettura oggetti distribuiti	6
6	ORB - Object Request Broker	6
6.1	Funzionamento	7
6.2	Vantaggi	7

7	Relazione tra prodotto e costo di produzione	7
7.1	Fattori principali	7
7.2	Equilibrio costi	7
8	Service Oriented Architecture	7
9	Secondo quali attività va suddivisa la fase di testing?	8
10	Come si chiama il testing di validazione per i software a contratto?	9
11	Ciclo di vita del Software	10
11.1	Definizione Formale	10
11.2	Stadi	10
11.2.1	Stadio 1: Sviluppo	10
11.2.2	Stadio 2: Manutenzione	10
11.2.3	Stadio 3: Dismissione	10
11.3	Fasi dello Stadio di Sviluppo	10
11.3.1	1. Requisiti	10
11.3.2	2. Specifiche	11
11.3.3	3. Pianificazione	11
11.3.4	4. Progetto	11
11.3.5	5. Codifica	12
11.3.6	6. Integrazione	12
12	Qual é la manutenzione più frequentemente utilizzata?	12
12.1	Altri tipi di manutenzioni	12
13	Regola 10-90	12
14	Stima durata progetto	13
15	Organizzazione modello di qualità del Software	13
15.1	Principi e struttura secondo IEEE 1061	13
15.1.1	1. Attività di Revisione	13
15.1.2	2. Attività di Transizione	14
15.1.3	3. Attività di Operazioni	14
15.1.4	Attributi di qualità	14
15.1.5	Processo iterativo di valutazione	15
16	Diagramma UML per interazione tra oggetti	15

17 COCOMO	16
17.1 Formula di COCOMO	16
17.1.1 Esempio	17
18 Differenze tra testing black/white box	17
18.1 Black Box	17
18.2 White Box	17
19 Affidabilità del Software	18
19.0.1 Caratteristiche dell'affidabilità	18
20 Specifica requisito di affidabilità.	18
21 Stima della dimensione del prodotto software	18
21.1 Analisi Bottom-Up	19
21.2 Tecniche di scomposizione	19
21.2.1 LOC (Lines-Of-Code)	19
21.2.2 FP (Function Points)	19
22 Che cos'è un Software Critico?	21
22.1 Caratteristiche	21
22.2 Esempi	21
23 Cosa si intende per durata di un prodotto software?	21
24 DRE	21
25 Design Pattern	22
25.1 Cosa sono i Design Patterns? a cosa servono?	22
25.2 Come è fatta la struttura di un Design Patterns?	22
25.3 Lista dei Design Pattern	23
25.3.1 Abstract Factory	23
25.3.2 Factory Method	24
25.3.3 Adapter	24
25.3.4 Composite	24
25.3.5 Decorator	25
25.3.6 Observer	25
25.3.7 Template Method	26
25.3.8 Strategy	26

1 Significato include/extend negli use case

Il diagramma degli use case permettere di rappresentare graficamente le interazioni tra attori e sistema, specificando quali casi d'uso gli attori possono attivare.

1.1 Include:

Indica che un caso d'uso include **obbligatoriamente** un altro caso d'uso per completare la sua funzionalità. Utilizzato quando il comportamento di un caso d'uso é riutilizzabile per altri casi d'uso. Caso d'uso principale richiede l'esecuzione dei casi d'uso inclusi.

1.2 Extend:

Indica che un caso d'uso può estendere **opzionalmente** un altro caso d'uso, aggiungendo funzionalità alternative o opzionali. Utilizzato per rappresentare casi che si verificano solo in determinate condizioni. L'attivazione del caso d'uso esteso dipende da una determinata condizione.

2 Requisiti Software

I requisiti del software sono la **descrizione dei servizi** che un sistema software deve fornire, insieme ai vincoli da rispettare sia in fase di sviluppo che durante la fase di operatività.

2.1 Requisiti utente:

Descrizione in **linguaggio naturale** dei **servizi** che il sistema deve fornire e dei **vincoli operativi**.

2.2 Requisiti di sistema:

Specificati mediante la **stesura di un documento strutturato** che descrive in modo dettagliato i **servizi che il sistema software deve fornire**. Si dividono in **3** categorie:

2.2.1 Funzionali:

Definiscono le funzionalità del sistema, ovvero i servizi offerti, le reazioni a specifici input e il comportamento in diverse situazioni operative.

2.2.2 Non funzionali:

Descrivono le qualità del sistema e del processo di sviluppo, come efficienza, affidabilità, sicurezza ed altre caratteristiche esterne.

2.2.3 Dominio:

Derivano dal contesto applicativo del sistema.

2.3 Differenze:

I requisiti funzionali in breve **definiscono** tutte le **operazioni** che il **sistema deve eseguire**. Gli altri **definiscono** tutte le **funzionalità** che il **sistema deve rispettare**.

3 Modello ciclo di vita

Definisce le fasi attraverso cui il prodotto software evolve, dalla raccolta dei requisiti alla dismissione e l'ordine in cui tali fasi vengono eseguite. La scelta del modello dipende da fattori come la natura e la maturità dell'organizzazione, le tecnologie adottate e i vincoli del cliente.

3.1 Principali modelli

3.1.1 Build & Fix

Modello privo di struttura formale; il prodotto viene sviluppato e poi modificato fino a soddisfare le esigenze del cliente.

3.1.2 Waterfall

Modello sequenziale a fasi successive con verifica alla fine di ogni fase; adatto a progetti con requisiti ben definiti.

3.1.3 Spirale

Modello iterativo che combina l'approccio del Waterfall con una forte enfasi sull'analisi dei rischi; ideale per progetti complessi e soggetti a cambiamenti nei requisiti.

4 Risk Analysis

É il processo di identificazione, valutazione e gestione dei potenziali problemi che possono compromettere il successo di un progetto software.

4.1 Fasi principali:

1. Identificazione dei rischi: Individuare tutte le minacce che possono influire negativamente sul progetto.
2. Valutazione dei rischi: Stimare l'impatto e la probabilità di ogni rischio.
3. Prioritizzazione: Stabilire quali rischi gestire per primi.
4. Pianificazione delle mitigazioni: Definire le azioni per ridurre la probabilità o l'impatto dei rischi.
5. Monitoraggio continuo: Assicurarsi che i rischi siano monitorati durante tutto il ciclo di vita del software.

5 Architettura softwares - Architettura oggetti distribuiti

L'architettura di sistema definisce la struttura dei componenti del sistema software, insieme alle relazioni tra questi ultimi. L'architettura ad oggetti distribuiti é un'architettura di sistema che non fa distinzioni tra client/server, infatti ogni oggetto distribuito può fungere sia da client che da server. La comunicazione remota fra gli oggetti é resa trasparente usando middleware basati sul concetto di software bus. Le applicazioni basate su questa architettura consistono in un insieme di oggetti che sono eseguiti su piattaforme distribuite ed eterogenee, e comunicano tramite invocazioni remote dei metodi.

6 ORB - Object Request Broker

É un componente middleware che abilita la comunicazione trasparente tra oggetti distribuiti su rete, indipendentemente da posizione fisica, linguaggio di programmazione o sistema operativo É il cuore delle architetture ad oggetti distribuiti, come CORBA.

6.1 Funzionamento

Intermediazione Richieste: il client invoca un metodo su un oggetto remoto; ORB localizza il server, inoltra la richiesta e restituisce il risultato. **Gestione Dati:** si occupa di serializzazione e deserializzazione dei parametri, oltre ai protocolli di rete **Trasparenze Fornite:** location transparency e implementation transparency.

6.2 Vantaggi

Permette agli sviluppatori di concentrarsi sulla logica applicativa, astraendo la complessità della distribuzione.

7 Relazione tra prodotto e costo di produzione

La relazione tra prodotto software e costo di produzione è prevalentemente positiva: maggiore è la dimensione, la complessità o la qualità del prodotto, maggiore è il costo iniziale di sviluppo, ma con impatti variabili sui costi a lungo termine.

7.1 Fattori principali

Dimensione del prodotto: La complessità e le funzionalità aumentano direttamente lo sforzo e i costi, spesso in modo non lineare. **Qualità:** Investire in qualità eleva i costi iniziali, ma riduce quelli futuri di manutenzione e correzioni. **Processi di sviluppo:** Metodologie efficienti e automazione abbassano i costi mantenendo la qualità

7.2 Equilibrio costi

Scalabilità e modularità: Favoriscono costi minori in fase di aggiornamenti e manutenzioni, bilanciando l'investimento iniziale. **Sintesi:** Esiste un trade-off tra costi di produzione upfront e spese post-rilascio, ottimizzabile con stime accurate basate su dimensioni funzionali.

8 Service Oriented Architecture

La SOA è un'architettura software distribuita che consiste in molteplici servizi. I Servizi sono distribuiti in maniera tale da poter essere eseguiti su

nodi differenti con differenti service provider. L'obiettivo é quello di sviluppare applicazioni software composte da servizi distribuiti, in modo tale che i singoli servizi possano essere eseguiti su piú piattaforme differenti e implementati con differenti linguaggi di programmazione. All'interno delle SOA troviamo ORB che permette la comunicazione tra i client e i servizi distribuiti.

Il fulcro dell'implementazione SOA sono i web services, e sfruttano vari protocolli per far comunicare i servizi fra di loro.

Tra questi troviamo: **SOAP**: protocollo basato su XML e HTML che permette lo scambio di informazioni in un sistema distribuito. Utilizza protocolli di trasporto come HTTP, SMTP. **REST**: architettura che utilizza i metodi HTML per operazioni CRUD.

Oltre che ai web services, le SOA utilizzano altri servizi, tra cui: **UDDI**: aiuta a localizzare e identificare i servizi presenti in una rete. **WSDL**: Linguaggio basato su XML utilizzato per descrivere le interfacce dei servizi in un'architettura di

9 Secondo quali attività va suddivisa la fase di testing?

La fase di testing é articolata in una serie di attività che assicurano la soddisfazione dei requisiti definiti. Queste attività sono:

1. Pianificazione del testing:

- Definizione degli obiettivi del testing.
- Identificazione delle risorse necessarie.
- Pianificazione temporale e stima dei costi.

2. Progettazione dei Test Case:

- Creazione di scenari di test basati sui requisiti funzionali e non funzionali.
- Identificazione delle versioni software da testare.

3. Preparazione dell'ambiente di test:

- Configurazione degli ambienti (Hardware, Software, Reti).
- Installazione delle versioni del software da testare.

4. **Esecuzione dei test:**

- Esecuzione dei casi di test progettati.
- Registrazione dei risultati (Successo o Fallimento) e documentazione delle anomalie.

5. **Analisi dei risultati:**

- Confronto tra risultati ottenuti e quelli attesi.
- Identificazione e classificazione dei difetti.

6. **Risoluzione dei problemi:**

- Collaborazione con il team di sviluppo per risolvere i difetti identificati.
- Verifica delle correzioni.

7. **Test di regressione:**

- Assicurarsi che le modifiche al codice non abbiano introdotto nuovi errori.

8. **Valutazione finale e report:**

- Verifica se il prodotto soddisfa i criteri di accettazione.
- Redazione di un report con le conclusioni del testing.

10 **Come si chiama il testing di validazione per i software a contratto?**

Il testing di validazione per i software a contratto é chiamato **Acceptance Testing**.

- **Definizione:** É l'ultima fase del ciclo di testing, in cui il cliente o l'utente finale verifica che il software soddisfi i requisiti contrattuali e sia pronto per il rilascio.
- **Scopo:** Validare che il software sia conforme alle specifiche contrattuali e garantire la piena utilizzabilità nel contesto operativo del cliente.

11 Ciclo di vita del Software

11.1 Definizione Formale

Il ciclo di vita di un software é il quadro strutturale che organizza in fasi, attività e compiti tutti i processi di sviluppo, esercizio e manutenzione di un prodotto software, dalla definizione dei requisiti fino alla dismissione del sistema. É suddiviso in 3 stadi, il primo stadio a sua volta é composto da 6 fasi.

11.2 Stadi

11.2.1 Stadio 1: Sviluppo

- Requisiti
- Specifiche
- Pianificazione
- Progetto
- Codifica
- Integrazione

11.2.2 Stadio 2: Manutenzione

Copre circa il 60% dei costi del ciclo di vita

11.2.3 Stadio 3: Dismissione

11.3 Fasi dello Stadio di Sviluppo

11.3.1 1. Requisiti

Obiettivo: Identificare e raccogliere le esigenze degli stakeholder. **Descrizione:** In questa fase si effettua la raccolta, l'analisi e la documentazione dettagliata delle esigenze degli utenti e degli stakeholder, per definire cosa il sistema deve fare e i suoi vincoli operativi. **Attività Principali:**

- Interviste con utenti e clienti.
- Analisi delle esigenze funzionali.
- Identificazione dei requisiti non funzionali.

- Creazione di una lista iniziale di requisiti.

11.3.2 2. Specifiche

Obiettivo: Documentare e dettagliare i requisiti raccolti nella fase precedente. **Descrizione:** Approfondisce i requisiti in specifiche tecniche dettagliate, producendo documenti come **SRS (Software Require Specification)** o **SFS (Software Functional Specification)** che serviranno nelle fasi successive. **Attività Principali:**

- Analisi di fattibilità tecnica ed economica.
- Stesura dei documenti di specifiche.
- Validazione dei requisiti con il cliente per verificare che siano corretti.

11.3.3 3. Pianificazione

Obiettivo: Organizzare il lavoro per il completamento del progetto. **Descrizione:** Si definisce il piano del progetto: stime di costi/risorse/tempi, allocazione team, scelta strumenti/metodologie, identificazione rischi e definizione milestone. Produce il **Project Plan** con **Gant**, budget e criteri di monitoraggio. È cruciale per allineare stakeholder e gestire scope.

11.3.4 4. Progetto

Obiettivo: Progettare l'architettura e i dettagli tecnici del software. **Descrizione:** Si definisce la struttura del sistema e le componenti, fornendo un modello chiaro da seguire nella fase di sviluppo. **Attività Principali:**

- Progettazione dell'architettura.
- Creazione dei diagrammi UML, diagrammi di flusso e design delle interfacce.
- Progettazione di database e definizione delle entità.
- Redazione del piano di test iniziale per assicurarsi che il progetto sia verificabile.

11.3.5 5. Codifica

Obiettivo: Tradurre il design in un software funzionante. **Descrizione:** In questa fase gli sviluppatori scrivono il codice sorgente seguendo le specifiche di progetto, standard di coding e best practice. Include unit testing durante lo sviluppo e commit in repository git. L'importanza di questa fase é per l'appunto la trasformazione da design a eseguibile, con enfasi su qualità e leggibilità.

11.3.6 6. Integrazione

Obiettivo: Collegare i diversi moduli e testare il sistema completo. **Descrizione:** SI assemblano i moduli/test unitari in un sistema coerente, verificando integrazioni tramite test di integrazione. Identifica e risolve difetti di interfaccia. Culmina in un prototipo parziale o completo pronto per essere testato.

12 Qual é la manutenzione piú frequentemente utilizzata?

La manutenzione piú frequentemente utilizzata é quella **correttiva**. La manutenzione correttiva é l'insieme delle attività volte a correggere difetti, errori o malfunzionamenti riscontrati nel software dopo il suo rilascio in produzione, ripristinando la funzionalità conforme alle specifiche originali.

12.1 Altri tipi di manutenzioni

1. Adattiva Modifiche al software per adattarlo ai cambiamenti dell'ambiente operativo.
2. Perfettiva Migliora le prestazioni o aggiunge nuove funzionalità per soddisfare le richieste degli utenti.
3. Preventiva Effettuare modifiche per ridurre rischi di guasto o per prevenire dei problemi futuri.

13 Regola 10-90

La regola 10-90 é una variante della regola 90-90, un aforismo umoristico dell'ingegneria del software. Questa regola afferma che: "Solo il 10% del

codice critico consuma il 90% delle risorse di esecuzione o sviluppo". Il 10% del codice in questione si definisce **core** del programma.

14 Stima durata progetto

Per stimare la durata totale di un progetto software partendo dal documento di specifica, è necessario utilizzare tecniche di pianificazione e stima che si basano su diversi fattori come la dimensione stimata del prodotto, le risorse disponibili e i processi di sviluppo. La prima fase della stima è quella stimare le dimensioni totali del software, usando tecniche note che vedremo avanti. Successivamente si stima la durata:

1. Modello algoritmico **COCOMO**:
 - Stimando effort e poi tempo
2. Usando metodi empirici:
 - Suddividere il progetto in attività specifiche e stimare la durata di ogni attività e successivamente sommare tutte queste stime per averne una complessiva.

15 Organizzazione modello di qualità del Software

Il modello di qualità del software si struttura principalmente attorno allo standard **IEEE 1061**, che fornisce un framework sistematico per definire, misurare e valutare la qualità attraverso metriche quantitative legate a obiettivi specifici del progetto.

15.1 Principi e struttura secondo IEEE 1061

Lo standard organizza la qualità del software identificando prima gli obiettivi di qualità rilevanti per il contesto. Successivamente, associa a questi obiettivi attributi di qualità misurabili e metriche per valutarli. Un elemento centrale sono gli indici di qualità, che aggregano più metriche in indicatori sintetici per monitorare le prestazioni complessive. Questi indici si dividono in tre gruppi principali, allineati al modello McCall.

15.1.1 1. Attività di Revisione

- **Manutenibilità**: effort richiesto per individuare e correggere un errore in un programma operativo.

- **Flessibilità:** effort per modificare un prodotto software già in uso.
- **Testabilità:** effort per testare il software e verificarne la correttezza funzionale.

15.1.2 2. Attività di Transizione

- **Portabilità:** effort per trasferire il software ad un nuovo ambiente hardware/software.
- **Riusabilità:** grado in cui parti del software possono essere riutilizzate in altre applicazioni.
- **Interoperabilità:** effort per integrare il software con altri sistemi.
- **Evolutività:** effort per aggiornarlo con nuovi requisiti.

15.1.3 3. Attività di Operazioni

- **Correttezza:** misura in cui il software soddisfa specifiche e obiettivi utente.
- **Affidabilità:** probabilità che svolga le funzioni previste con la precisione richiesta.
- **Efficienza:** risorse di calcolo e codice necessarie per eseguire una funzione.
- **Integrità:** protezione da accessi non autorizzati a software o dati.
- **Usabilità:** effort per imparare, usare, preparare input e interpretare output.

15.1.4 Attributi di qualità

Oltre agli indici, **IEEE 1061** elenca attributi fondamentali, come:

- **Complessità:** comprensibilità e verificabilità degli elementi e delle loro interazioni.
- **Precisione:** accuratezza dei calcoli e dei risultati.
- **Completezza:** implementazione esaustiva delle funzionalità richieste.
- **Coerenza:** uniformità di design, implementazione e notazioni.

- **Tolleranza:** capacità di operare in condizioni avverse.
- **Tracciabilità:** legami chiari tra elementi del processo di sviluppo.
- **Espandibilità:** possibilità di ampliare memorie o funzioni.
- **Generalità:** ampiezza delle applicazioni potenziali.
- **Modularità:** indipendenza dei moduli.
- **Autodocumentazione:** presenza di documentazione inline.

Questi attribuit sono mappati su metriche concrete

15.1.5 Processo iterativo di valutazione

IEEE 1061 definisce un ciclo chiaro:

1. **Definizione degli obiettivi:** contestualizzati al progetto.
2. **Identificazione degli attributi:** selezione di quelli prioritari.
3. **Scelta metriche:** quantitative e rilevanti per ciascun attributo.
4. **Misurazione:** calcolo e confronto con benchmark o target.
5. **Analisi e Feedback:** miglioramento continuo basato sui risultati.

Questa struttura garantisce che la qualità sia oggettiva, misurabile e allineata agli stakeholder, riducendo soggettività e rischi.

16 Diagramma UML per interazione tra oggetti

Per specificare l'interazione tra oggetti, usando la struttura dei diagrammi UML, possiamo usare il **Sequence Diagram**. Un altro diagramma UML che permette di fare questa cosa si chiama **Collaboration Diagram**. Questo diagramma è una rappresentazione equivalente del sequence diagram, infatti a partire da uno possiamo generare l'altro e viceversa. Il funzionamento del Collaboration Diagram è identico a quello del sequence diagram, cambia solo:

- Il Sequence Diagram descrive lo scambio di messaggi tra oggetti in ordine temporale, mentre il Collaboration Diagram descrive lo scambio di messaggi tra oggetti mediante relazioni.
- I Sequence vengono usati in fase di OOA, mentre i Collaboration in fase di OOD

17 COCOMO

Modello algoritmico introdotto da Boehm per determinare il valore dell'effort in base alla dimensione del prodotto. Il valore dell'effort viene successivamente utilizzato per determinare durata e costi di sviluppo. COCOMO comprende 3 modelli:

- **Basic:** Per stime iniziali.
- **Intermediate:** Usato dopo aver suddiviso il sistema in sottosistemi.
- **Advanced:** Usato dopo aver suddiviso in moduli ciascun sottosistema

La stima dell'effort viene effettuata a partire da:

- Stima delle dimensioni del progetto in **KLOC**
- Stima del modo di sviluppo del prodotto, che misura il livello intrinseco di difficoltà nello sviluppo, tra:
 - **Organic:** Per prodotti di piccole dimensioni.
 - **Semi-Detached:** Per prodotti di dimensioni intermedie.
 - **Embedded:** Per prodotti complessi.

17.1 Formula di COCOMO

Stima dell'effort nominale:

$$EffortNom = a \cdot (KLOC)^b \quad (1)$$

a, b sono coefficienti specifici per ciascuna categoria di progetto.

Stima effort:

$$Effort = EffortNom \cdot C \quad (2)$$

Dove C è il costo driver multiplier, vedi sotto.

Stima del tempo:

$$Time = c \cdot (Effort)^d \quad (3)$$

c, d come a, b

17.1.1 Esempio

MM = uomo/mesi, quante persone per mese devono lavorare al progetto
Modello intermediate, modo organic:

- **Passo 1:** Determinare l'effort nominale con la formula (1)

$$3.2 \cdot (33)^{1.05}$$

- **Passo 2:** Ottenere la stima dell'effort applicando un fattore moltiplicativo *C* (Cost Driver Multiplier) basato su *x* cost drivers. Usare formula (2)

$$126 \cdot 1.15 = 145MM$$

- **Passo3:** Stima del tempo della consegna. USare formula (3)

$$T = 2.5 \cdot 145^{0.38} \sim 16.56$$

18 Differenze tra testing black/white box

Sono due approcci per il testing di software, ma si differenziano per il loro focus e il livello di conoscenza da parte del tester.

18.1 Black Box

Approccio in cui il software viene considerato come una scatola nera. I Test Case vengono derivati dalle specifiche del software. Il Tester presenta gli input al componente o al sistema ed esamina l'output corrispondente. Se gli output non sono quelli specificati, allora il test ha rilevato un problema all'interno del software. Questo test viene spesso chiamato **Testing Funzionale**, perché il tester si concentra solo sulle funzionalità del Software e non considera per niente l'implementazione interna del software.

18.2 White Box

Il testing strutturale in cui si va a testare la struttura interna del codice. Si testano tutti i flussi di esecuzione. Se alcuni flussi non vengono eseguiti o vengono eseguiti in maniera errata, allora il software presenta dei problemi. Si usa per rendere efficiente e pulire il codice.

19 Affidabilità del Software

In maniera informale, l'affidabilità del software é la credibilità del software mentre in maniera formale é la probabilità che il prodotto software lavori "correttamente" in un determinato intervallo temporale. Intuitivamente un prodotto software é inaffidabile se presenta molti difetti. L'affidabilità migliora quando si riducono i difetti.

19.0.1 Caratteristiche dell'affidabilità

É una relazione non semplice tra:

- Affidabilità osservata,
- Numero di difetti latenti.
- Eliminare difetti nella parte di prodotto raramente utilizzata migliora l'affidabilità di un fattore molto piccolo.
- Il miglioramento dell'affidabilità per l'eliminazione del difetto dipende da quale porzione di sistema contiene quel difetto (Regola 10-90). Se il difetto é presente nel 10% di software piú usato dall'utenza, allora l'affidabilità aumenterà di molto.
- Quindi l'affidabilità Software dipende da come é usato quel prodotto.
- Un utente che usa una porzione di software può riscontrare un difetto che un altro utente che non usa quella porzione non ha riscontrato.

QUindi si afferma che l'affidabilità del software dipende dall'utenza che utilizza quel software.

20 Specifica requisito di affidabilità.

I requisiti di affidabilità fanno parte dei requisiti

21 Stima della dimensione del prodotto software

Per stimare la dimensione del prodotto software a partire dal documento di specifica vengono utilizzate diverse tecniche che analizzano i requisiti del sistema e cercano di quantificare la dimensione in termini di linee di codice, funzionalità, o altri parametri. Tra i metodi principali troviamo:

21.1 Analisi Bottom-Up

Che include:

- **FPA (Function Points Analysis)**
- **LOC (Lines Of Codes)**
- **Algoritmi Empirici**
- Stime su progetti già completati

21.2 Tecniche di scomposizione

Le tecniche di scomposizione utilizzano una strategia "divide et impera" e sono basate su:

- Stime dimensionali, come **LOC** o **FP**
- Suddivisione dei task e/o delle funzioni con relativa stima di allocazione dell'effort

21.2.1 LOC (Lines-Of-Code)

LOC stima il numero complessivo di linee di codice all'interno dell'intero software

21.2.2 FP (Function Points)

FP é una misura pesata delle funzionalità del software. Misura l'ammontare di funzionalità all'interno del sistema basandosi sulle specifiche dello stesso.

Si calcola in due step:

1. Calcolare **UFC** (Unadjusted Function point Count)
2. Moltiplicare **UFC** per il **TCF** (Technical Complexity Factor)

$$FP = UFC \cdot TCF$$

1. **UFC** (Unadjusted Function point Count)

É il risultato della somma ponderata di 5 componenti, ognuno classificato per la complessità.

- (a) **EI (External Input):** Input esterni che attraversano l'applicazione.

- (b) **EO (External Output)**: Output derivati che escono dal sistema.
- (c) **EQ (External Inquiries)**: Richieste di Interrogazione (Input + Output senza derivazioni).
- (d) **ILF (Internal Logic Files)**: File di dati logicamente correlati, interni all'applicazione.
- (e) **EIF (External Interface Files)**: File di dati esterni, referenziati ma non mantenuti.

Ogni componente ha dei pesi definiti da 3 livelli di complessità:

Componente	Low	Average	High
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

$$UFC = \sum_{i=1}^5 (\text{Numero Componenti} \cdot \text{Peso}_i)$$

2. TCF (Technical Complexity Factor)

Esistono 15 diversi fattori chiamati **Fattori di Degree of Influence**. Ad ogni fattore viene associato un valore intero compreso fra:

- 0 (**Irrilevante**)
- 5 (**Essenziale**)

Una volta stabilita la rilevanza di ogni fattore, il **TCF** viene calcolato:

$$TCF = 0.65 + 0.01 \sum_{j=1}^{14} F_j$$

Il **TCF** varia da 0.65 (se tutti i fattori F_j sono irrilevanti) a 1.35 (se tutti i fattori F_j sono essenziali), quindi l'adjustment é ± 35 %

22 Che cos'è un Software Critico?

Un software critico è un sistema software la cui malfunzione può avere gravi conseguenze, come la perdita di vite umane, danni significativi a infrastrutture o elevati costi economici.

22.1 Caratteristiche

- **Alta Affidabilità:** Deve funzionare correttamente in tutte le condizioni previste.
- **Sicurezza:** Deve essere protetto da guasti, attacchi e accessi non autorizzati.
- **Performance rigorose:** Spesso deve operare in tempo reale.

22.2 Esempi

- Software per dispositivi medici
- Software per controllo del traffico aereo
- Sistemi finanziari critici
- Sistemi di controllo industriale o infrastrutture energetiche

23 Cosa si intende per durata di un prodotto software?

È il periodo di tempo durante il quale il prodotto software è considerato utile, operativo e rilevante per soddisfare i bisogni degli utenti o degli stakeholder. Quindi la durata altro non è che il ciclo di vita stesso del prodotto.

24 DRE

Defect removal efficiency è un rapporto:

$$DRE = \frac{\text{bug fissati durante lo sviluppo software}}{\text{bugtotali}}$$

Serve per vedere quanto uno sviluppatore lavora al codice. $DRE > 92\%$ sviluppatore lavora bene.

25 Design Pattern

25.1 Cosa sono i Design Patterns? a cosa servono?

Uno dei compiti piú difficili per un progettista di software é individuare un insieme di oggetti ben definiti, riusabili il piú possibile e con relazioni e gerarchie chiare ed efficaci. I Design Patterns sono proprio la soluzione a questo problema. Questi trovano il fulcro della soluzione ad un problema ricorrente e fanno in modo che possa essere riutilizzata infinite volte anche non applicandola necessariamente nello stesso modo. In particolare essi creano un linguaggio condiviso tra gli sviluppatori in modo tale da offrire soluzioni a problemi che ritornano spesso nello sviluppo software e creano un codice strutturato in modo corretto. Inoltre capitalizzano l'esperienza della progettazione OO e ne favoriscono il riuso. Le soluzioni che propongono non sono mai già note ma allo stesso tempo non sono neanche soluzioni universali e quindi non sono sempre applicabili a qualsiasi problema.

I Pattern si distinguono in tre categorie principali:

- **Creazionali:** Riguardano la creazione degli oggetti.
- **Strutturali:** Si concentrano sull'organizzazione di classi e oggetti.
- **Comportamentali:** Descrivono come gli oggetti interagiscono e come si distribuiscono le responsabilità.

25.2 Come é fatta la struttura di un Design Patterns?

Un Design Patterns ha la seguente struttura: Per prima cosa troviamo il nome e la classificazione che illustrano l'essenza, lo scopo e il raggio d'azione del Pattern. In seguito troviamo la motivazione ovvero una descrizione del problema e dello scenario al quale bisogna applicare il relativo Pattern. In seguito troviamo l'applicazione del Pattern e subito dopo la struttura cioè una descrizione grafica della configurazione degli elementi che risolvono il problema. Troviamo anche le classi, gli oggetti che ne fanno parte e i risultati dopo l'applicazione di tale Pattern (conseguenze). Sono presenti poi alcune tecniche e consigli per applicare il Pattern (implementazione). Infine troviamo dei codici d'esempio che illustrano come implementare il Pattern in un certo linguaggio, degli esempi di applicazioni reali che lo utilizzano e altri Pattern che sono collegati a questo.

- **Nome e Classificazione:** Il nome illustra l'essenza di un Pattern; la classificazione lo identifica in termini di scopo e raggio d'azione.

- **Motivazione:** Scenario che descrive in modo astratto il problema al quale applicare il Pattern.
- **Applicabilità:** Applicazioni del Pattern.
- **Struttura:** Descrive graficamente la configurazione di elementi che risolvono il problema.
- **Partecipanti:** Classi ed oggetti che fanno parte del Pattern.
- **Conseguenze:** Risultati dopo l'applicazione del Pattern.
- **Implementazione:** Tecniche e suggerimenti per applicare il Pattern.
- **Codice di esempio:** Frammenti di codice che illustrano come implementare in un certo linguaggio di programmazione il Pattern.
- **Usi noti:** Esempi di applicazioni in sistemi reali.
- **Pattern collegati:** Altri Pattern.

25.3 Lista dei Design Pattern

25.3.1 Abstract Factory

L'Abstract Factory é un design pattern creazionale che fornisce un'interfaccia per creare famiglie di oggetti correlati senza specificare le loro classi concrete. Ogni factory concreta produce un'intera suite di prodotti coerenti tra loro.

1. Struttura

- **AbstractFactory:** Dichiarare i metodi per creare ogni tipo di prodotto della famiglia.
- **Factory concrete:** Implementano tutti i metodi della Abstract Factory ritornando prodotti specifici della famiglia.
- **Interfacce Astratte:** Per ogni tipo di prodotto.
- **Client:** Usa la factory per ottenere intere famiglie di prodotti coerenti.

25.3.2 Factory Method

Il Factory Method é un design pattern creazionale che definisce un'interfaccia per creare oggetti, ma delega alle sottoclassi la responsabilit  di decidere quale classe concreta istanziare.

1. Struttura

- **Creator:** Classe che definisce il metodo `factoryMethod()`.
- **ConcreteCreator:** Sottoclasse che implementa il metodo `factoryMethod()`.
- **ConcreteProduct:** Oggetto concreto creato dal metodo `factoryMethod()`.
- **Product:** Rispettiva classi di `ConcreteProduct`.

25.3.3 Adapter

L'Adapter é un design pattern strutturale che consente a due interfacce incompatibili di lavorare insieme, permettendo a oggetti con interfacce diverse di collaborare senza modificare il loro codice sorgente.

1. Struttura

- **Target Interface:** L'interfaccia che il client si aspetta.
- **Adaptee:** La classe esistente con interfaccia incompatibile che deve essere utilizzata.
- **Adapter:** Implementa il Target e "wrappa" l'adaptee, traducendo le chiamate da target ad adaptee.
- **Client:** Usa l'adapter senza sapere che dietro c'  l'adaptee.

2. Approcci Implementativi

- **Class Adapter:** Usa ereditariet .
- **Object Adapter:** Usa composizione.

25.3.4 Composite

Il Composite é un design pattern strutturale che consente di comporre oggetti in strutture ad albero per rappresentare gerarchie parte-tutto. Permette ai client di trattare oggetti individuali e composizioni di oggetti allo stesso modo, offrendo un'interfaccia uniforme.

1. Struttura

- **Component:** Definisce l'interfaccia comune per foglie e composti.
- **Leaf:** Rappresenta gli elementi finali della gerarchia.
- **Composite:** Rappresenta i nodi interni che contengono altri componenti.
- **Client:** Lavora solo con interfacce Component, senza sapere se sta lavorando con foglia o composto

25.3.5 Decorator

Il Decorator é un design pattern strutturale che consente di aggiungere responsabilità a un oggetto dinamicamente, offrendo un'alternativa flessibile all'ereditarietà per estendere la funzionalità. Mantiene la stessa interfaccia dell'oggetto originale mentre aggiunge nuovi comportamenti.

1. Struttura

- **Component:** Definisce l'interfaccia per gli oggetti a cui aggiungere responsabilità.
- **ConcreteComponent:** L'oggetto base a cui aggiungiamo funzionalità.
- **Decorator:** Implementa Component e ottiene un'istanza di component.
- **ConcreteDecorator:** Estende Decorator e aggiunge specifiche responsabilità.
- **Client:** Lavora con l'interfaccia Component senza sapere se sta usando un componente base o un decorator.

25.3.6 Observer

L'Observer é un design pattern comportamentale che definisce una dipendenza uno-a-molti tra oggetti, in modo che quando un oggetto cambia stato, tutti gli oggetti dipendenti vengono notificati automaticamente e aggiornati. Implementa un meccanismo di sottoscrizione/notifica.

1. Struttura

- **Subject:** Mantiene una lista di observer e notifica il cambiamento di stato.

- **Observer:** Interfaccia che definisce come ricevere notifiche.
- **ConcreteObserver:** Implementa Observer e reagisce alle notifiche del Subject.
- **ConcreteSubject:** Mantiene lo stato e notifica gli Observers quando cambia.
- **Client:** Registra gli Observers con il Subject.

25.3.7 Template Method

Il Template Method é un design pattern comportamentale che definisce la struttura (sequenza di passi) di un algoritmo in una superclasse, permettendo alle sottoclassi di ridefinire alcuni passi specifici senza modificare l'ordine generale dell'algoritmo.

1. Struttura

- **AbstractClass:** Definisce il `templateMethod()` (Algoritmo) con alcuni passi implementati ed altri lasciati astratti per la personalizzazione
- **templateMethod:** Struttura dell'algoritmo che chiama i passi in ordine.
- **abstractMethod:** Metodi definiti nell'AbstractClass che vengono utilizzati e modificati
- **ConcreteSublasses:** Forniscono l'implementazione dei metodi astratti, personalizzando step specifici ma preservando l'algoritmo standard

25.3.8 Strategy

Lo Strategy é un design pattern comportamentale che definisce una famiglia di algoritmi, li incapsula ciascuno in una propria classe e li rende intercambiabili, permettendo al contesto di variare il comportamento a runtime senza cambiare il suo codice.

1. Struttura

- **Strategy:** Definisce l'operazione astratta, ad esempio `'execute()'` o `'calculate()'`, comune a tutte le strategie.
- **ConcreteStrategyA/B:** Implementano la strategy con algoritmi diversi ma stessa interfaccia.

- **Context:** Contiene riferimenti a una Strategy e delega a essa il comportamento variabile.
- **Client:** É responsabile della selezione della strategy appropriata in base al contesto